

TECH

CODE REVIEW / ENGINEERING M

CODE REVIEW

CODE REVIEW

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК общая практика индустрии

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 ЛЁГКИЙ	ОБЩИЙ ПОДХОД Q1 · Q2 · Q3	BLOCKER vs SUGGESTION	С чего начать ревью и как маркировать комментарии.
ДЕНЬ 2 СРЕДНИЙ	ТОН SPOR Q4 · Q5 · Q6	КОГДА СОЗВОН	Дать жёсткий feedback без токсичности.
ДЕНЬ 3 СРЕДНИЙ	АРХИТЕКТУРА Q7 · Q8 · Q9	HOOKS TS	На что смотреть в трёх частых типах PR ·
ДЕНЬ 4 СРЕДНИЙ	PERFORMANCE Q10 · Q11 · Q12	SECURITY A11Y	Три аудита, которые часто упускают.
ДЕНЬ 5 СРЕДНИЙ	TESTS БОЛЬШОЙ PR Q13 · Q14 · Q15	РАЗБИВКА	Тесты и борьба с PR-монстрами.
ДЕНЬ 6 СРЕДНИЙ	RUBBER-STAMP Q16 · Q17 · Q18	CHECKLIST КАЧЕСТВО	Как поднять планку без формализма.
ДЕНЬ 7 СРЕДНИЙ	LEGACY ПЕРФЕКЦИОНИЗМ Q19 · Q20		Не блокировать delivery, но не пропускать долг.

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА ЛЁГКИЙ 3 ВОПРОСА

ОБЩИЙ ПОДХОД BLOCKER vs SUGGESTION**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

С чего начать ревью и как маркировать комментарии.

ВОПРОСЫ ДНЯ

Q01 На что ты смотришь в code review?

Q02 Что для тебя blocker comment?

Q03 Что для тебя non-blocking suggestion?

На что ты смотришь в code review?

ОПИСАНИЕ

Иерархия: 1) корректность (делает ли то, что заявлено), 2) безопасность и риски, 3) тесты, 4) читаемость и архитектура, 5) перфоманс и ally, 6) стиль (если линтер не покрывает). Сначала большое – потом мелочи.

КАК РАБОТАЕТ

- 01 Прочитать описание PR и issue.
- 02 Понять scope изменений – diff целиком.
- 03 Запустить локально, если PR нетривиальный.
- 04 Комментарии маркировать (blocker / suggestion / nit).
- 05 Не размазывать – главное в начале.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR.
- 02 Особенно – критичные сценарии (auth, payment).
- 03 PR от новых разработчиков – учить, не наказывать.

ПОДВОДНЫЕ КАМНИ

- 01 Только nitpicks без архитектурных вопросов.
- 02 Игнорировать тесты.
- 03 Ревью только diff без context.

МОГУТ ДОСПРОСИТЬ

- 01 Что для тебя blocker?
- 02 В каком порядке смотреть PR?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/>

ВОПРОС Q02

Что для тебя blocker comment?

ОПИСАНИЕ

Blocker – то, что должно быть исправлено перед merge: корректность сломана (баг), безопасность (XSS, утечка PII), регрессия перфоманса, нарушение архитектурных границ, отсутствующие тесты на критичный путь.

КАК РАБОТАЕТ

- 01 Маркировать явно: 'Blocker:' / 'Must fix:'.
- 02 Объяснять причину – не «исправь», а «иначе сломается X».
- 03 Предлагать решение, если очевидно.
- 04 Если блокер сложный – созвониться (см. Q6).

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Bugs, security, регрессии.
- 02 Нарушение публичного API.
- 03 Отсутствие тестов на критичный сценарий.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много blocker = демотивация автора.
- 02 Blocker по личному вкусу – не blocker.
- 03 Без объяснения – выглядит как «нет потому что нет».

МОГУТ ДОСПРОСИТЬ

- 01 Чем blocker от suggestion отличается?
- 02 Кто решает спор о blocker?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/standard.html>

Что для тебя non-blocking suggestion?

ОПИСАНИЕ

Suggestion / nit – улучшение, не критичное для merge. Маркировать явно ('nit:', 'suggestion:'). Автор сам решает, брать или отложить. Это снимает напряжение и ускоряет PR.

КАК РАБОТАЕТ

- 01 'nit:' для микро-косметики.
- 02 'suggestion:' для альтернативного подхода.
- 03 'fyi:' для информации без действия.
- 04 В одном PR – не больше 2-3 значимых suggestions.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Косметические правки имени переменной.
- 02 Альтернативный, но не лучший паттерн.
- 03 Идеи на будущее.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком много nit – захламляют PR.
- 02 Suggestion без 'nit:' – автор воспринимает как blocker.
- 03 Споры по nit – пустая трата времени.

МОГУТ ДОСПРОСИТЬ

- 01 Когда nit = blocker?
- 02 Как отделить вкус от объективного качества?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/comments.html>

ДЕНЬ 2 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

ТОН СПОР КОГДА СОЗВОН**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Дать жёсткий feedback без токсичности.

ВОПРОСЫ ДНЯ

Q04 Как давать жёсткий feedback без токсичности?

Q05 Как спорить о решении в PR?

Q06 Когда лучше созвониться, а не писать комментарии?

Как давать жёсткий feedback без токсичности?

ОПИСАНИЕ

Принципы: критика КОДА, не автора; объяснять «почему», не только «что»; предлагать альтернативу; признавать хорошее наряду с плохим; писать как себе. Тон важнее содержания.

КАК РАБОТАЕТ

- 01 «Этот код делает X, что плохо, потому что Y. Можно сделать Z».
- 02 Не «ты сделал», а «здесь сделано».
- 03 Хвалить хорошие куски явно.
- 04 Тон – нейтральный, без эмодзи-сарказма.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Каждое review.
- 02 Особенно – junior / новички.
- 03 PR от мейнтейнеров – где есть субординация.

ПОДВОДНЫЕ КАМНИ

- 01 Сарказм воспринимается тяжелее, чем кажется автору.
- 02 «Тебе бы стоило» – пассивная агрессия.
- 03 Обвинение → конфликт → blocked PR.

МОГУТ ДОСПРОСИТЬ

- 01 Как реагировать на агрессию автора?
- 02 Что делать при множественных blocker?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/comments.html>

ВОПРОС Q05

Как спорить о решении в PR?

ОПИСАНИЕ

Использовать данные и стандарты команды, не вкус. Если спор затягивается – выносить в синхрон (созвон, документ ADR), а не множить треды. У спора должен быть тайм-боксинг.

КАК РАБОТАЕТ

- 01 Опирается на: код-стандарты, ADR, прошлые decisions.
- 02 Не побеждать, а найти лучшее решение.
- 03 Если 2-3 итерации не сходятся – синхрон.
- 04 Записать решение, чтобы не повторять.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Архитектурные споры в PR.
- 02 Конфликт стиль / читаемость.
- 03 Trade-off безопасности vs скорости.

ПОДВОДНЫЕ КАМНИ

- 01 Спор затягивается – PR блокируется на дни.
- 02 Личное эго побеждает аргументы.
- 03 Без записи – спор повторяется в следующий PR.

МОГУТ ДОСПРОСИТЬ

- 01 Кто арбитр в споре?
- 02 Когда писать ADR?

ПОЛНАЯ СТАТЬЯ

<https://adr.github.io/>

Когда лучше созвониться, а не писать комментарии?

ОПИСАНИЕ

Когда: длинный спор (>3 итераций); большая архитектурная правка с trade-offs; эмоциональный накал; объяснить сложный контекст. 10 минут созвона экономят часы тредов.

КАК РАБОТАЕТ

- 01 Считать треды: > 3 ответов = созвон.
- 02 Сразу записывать решения.
- 03 Pair-review за общим экраном.
- 04 Async для маленьких PR, sync – для крупных.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Архитектурные refactor.
- 02 Multi-step миграция.
- 03 Когда тон становится острым.

ПОДВОДНЫЕ КАМНИ

- 01 Slack-DM без записи – решение теряется.
- 02 Avoid sync as default – перегруз календарей.

МОГУТ ДОСПРОСИТЬ

- 01 Как фиксировать решения после созвона?
- 02 Кого приглашать?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/comments.html>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

АРХИТЕКТУРА HOOKS TS**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

На что смотреть в трёх частых типах PR.

ВОПРОСЫ ДНЯ

- Q07 Как ревьюить архитектурные изменения?
- Q08 Как ревьюить React hooks?
- Q09 Как ревьюить TypeScript types?

Как ревьюить архитектурные изменения?

ОПИСАНИЕ

Перед кодом – design-doc / ADR; смотреть на согласованность со слоями (FSD / Clean), правила импортов, public API; оценить trade-offs (performance, поддерживаемость, riski); проверить миграционный план для legacy-кода.

КАК РАБОТАЕТ

- 01 Запросить design-doc до PR.
- 02 Сначала структура, потом imp-файлы.
- 03 Проверить нарушения слоёв.
- 04 Импорт направление: feature → entity → shared.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Новый слой / модуль.
- 02 Большой refactor.
- 03 Adoption нового state-manager / library.

ПОДВОДНЫЕ КАМНИ

- 01 Code-review без design-фазы – поздно решать.
- 02 Цикл feature → feature – нарушение архитектуры.
- 03 Большой PR без stages.

МОГУТ ДОСПРОСИТЬ

- 01 Что должно быть в design-doc?
- 02 Какие линтеры проверяют слои?

ПОЛНАЯ СТАТЬЯ

<https://feature-sliced.design/>

ВОПРОС Q08

Как ревьюить React hooks?

ОПИСАНИЕ

Чек-лист: правильные deps в `useEffect` / `useMemo` / `useCallback` (`eslint-plugin-react-hooks/exhaustive-deps`), нет stale closure, `cleanup` для подписок и таймеров, нет `setState` в `render`, `useState` нужного гранулярности (не один объект на 10 полей).

КАК РАБОТАЕТ

- 01 `exhaustive-deps` включён в линтере.
- 02 `Effects: cleanup return`.
- 03 `useMemo` / `useCallback` с правильными deps.
- 04 Custom hooks: одна ответственность.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR с хуками.
- 02 Кастомные хуки — особенно внимательно.
- 03 Эффекты с подписками.

ПОДВОДНЫЕ КАМНИ

- 01 Игнорированный `exhaustive-deps` без комментария.
- 02 `Cleanup` забыт — утечка.
- 03 `useState` с большим объектом — лишние ререндеры.

МОГУТ ДОСПРОСИТЬ

- 01 Что делать со stale closure?
- 02 Когда `useReducer` лучше `useState`?

ПОЛНАЯ СТАТЬЯ

<https://react.dev/reference/react/hooks>

Как ревьюить TypeScript types?

ОПИСАНИЕ

Маркеры плохого TS: `any` / `as` без обоснования, неоправданно сложные `generic`-цепочки, дубль типа и `Zod`-схемы (расходятся), `@ts-ignore` без `TODO`, мутация типов через `as`. Типы должны быть простыми и понятными.

КАК РАБОТАЕТ

- 01 `any` → `unknown` + `guard`.
- 02 `as` → `type guard` / `Zod` / `narrowing`.
- 03 `@ts-ignore` → `@ts-expect-error` с `TODO`.
- 04 `Generic` ≤ 2-3 параметров.
- 05 Никаких импликаций `any`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой TS PR.
- 02 Особенно – `generic` утилиты.
- 03 Adoption `Zod` / новых типов.

ПОДВОДНЫЕ КАМНИ

- 01 Лишние `generic` – никто не понимает.
- 02 `as User` без проверки – `runtime`-баги.
- 03 Дублирование TS-тип + `Zod` – рассинхрон.

МОГУТ ДОСПРОСИТЬ

- 01 Какие правила TS-стайла обязательны?
- 02 Когда `as` ок?

ПОЛНАЯ СТАТЬЯ

<https://typescript-eslint.io/rules/>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

PERFORMANCE SECURITY A11Y**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три аудита, которые часто упускают.

ВОПРОСЫ ДНЯ

Q10 Как ревьюить performance-sensitive code?

Q11 Как ревьюить security-sensitive code?

Q12 Как ревьюить accessibility?

Как ревьюить performance-sensitive code?

ОПИСАНИЕ

Спрашивать «есть ли measurement?» — без бенчмарка трудно судить. Смотреть: ненужные re-render, тяжёлые операции в render, отсутствие виртуализации в больших списках, лишние allocations в горячем пути, корректность useМемо / useCallback (с правильными deps).

КАК РАБОТАЕТ

- 01 Запросить before/after метрики.
- 02 Profiler-trace или Lighthouse-отчёт.
- 03 Проверить render-cascade в DevTools.
- 04 Bundle-size diff.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Дашборды.
- 02 Big tables / lists.
- 03 Hot paths (search, scroll).

ПОДВОДНЫЕ КАМНИ

- 01 Преждевременная оптимизация без данных.
- 02 useМемо на всё — overhead.
- 03 Микрооптимизации в холодных путях.

МОГУТ ДОСПРОСИТЬ

- 01 Какие метрики просить?
- 02 Когда мемоизация лишняя?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/vitals>

ВОПРОС Q11

Как ревьюить security-sensitive code?**ОПИСАНИЕ**

Чек-лист: dangerouslySetInnerHTML без sanitization, unescaped user input в URL / DOM, CSP-violation, secrets в коде / env / bundle, токены в localStorage, CORS-настройки, отсутствие server-side проверки auth.

КАК РАБОТАЕТ

- 01 grep dangerouslySetInnerHTML – sanitization?
- 02 Audit deps: npm audit, Snyk.
- 03 Token storage: cookie httpOnly vs localStorage.
- 04 CSP / Permissions-Policy headers.
- 05 Server-side auth-check – обязательно.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth flows.
- 02 Payment / transfer.
- 03 User-generated content.
- 04 Admin actions.

ПОДВОДНЫЕ КАМНИ

- 01 Frontend-only auth check.
- 02 Token in localStorage без HttpOnly cookie.
- 03 User-input в template literal SQL/HTML.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CSP?
- 02 Где хранить refresh-token?

ПОЛНАЯ СТАТЬЯ

<https://owasp.org/www-project-top-ten/>

Как ревьюить accessibility?

ОПИСАНИЕ

Чек-лист: semantic HTML (button vs div), accessible name (label / aria-label), focus management, keyboard navigation, ARIA только когда нужно, контраст, reduced-motion. Тестировать через axe / Lighthouse ally / screen reader.

КАК РАБОТАЕТ

- 01 Native button > div role=button.
- 02 Label всегда у input.
- 03 Focus-trap в modal.
- 04 axe-core CLI / browser extension.
- 05 Lighthouse ally score 90+.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom interactive components.
- 02 Forms.
- 03 Modals / dropdowns.
- 04 Tables.

ПОДВОДНЫЕ КАМНИ

- 01 Tooltip без aria-describedby.
- 02 Click-only handlers без keyboard.
- 03 ARIA-теги на всё.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое axe?
- 02 Когда ARIA вреден?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/standards-guidelines/aria/>

ДЕНЬ 5 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

TESTS БОЛЬШОЙ PR РАЗБИВКА**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Тесты и борьба с PR-монстрами.

ВОПРОСЫ ДНЯ

- Q13 Как ревьюить tests?
- Q14 Как понять, что PR слишком большой?
- Q15 Как разбить большой PR?

Как ревьюить tests?

ОПИСАНИЕ

Тесты тоже код — должны быть читаемы и поддерживаемы. Чек-лист: тестируем поведение, не implementation; осмысленные имена; Arrange-Act-Assert; нет flaky-сторон (времени, рандома); mock минимальный; coverage не самоцель.

КАК РАБОТАЕТ

- 01 Имена тестов: 'should X when Y'.
- 02 Arrange-Act-Assert — три зоны.
- 03 Mock external — не internal.
- 04 Avoid implementation details (data-testid).
- 05 MSW для API mocks.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR с тестами.
- 02 Особенно — flaky tests audit.
- 03 Принятие новой фичи.

ПОДВОДНЫЕ КАМНИ

- 01 Coverage 100% — но без поведения.
- 02 Mock всего — тест ничего не доказывает.
- 03 data-testid вместо getByRole.

МОГУТ ДОСПРОСИТЬ

- 01 Какие тесты не стоит писать?
- 02 Чем MSW лучше jest.fn?

ПОЛНАЯ СТАТЬЯ

<https://testing-library.com/docs/queries/about/#priority>

ВОПРОС Q14

Как понять, что PR слишком большой?**ОПИСАНИЕ**

Эмпирика: > 400 строк изменений, > 10 файлов, несколько unrelated концернов. Reviewer не может качественно прочитать большой PR – пропускает баги. Лучше разбить (см. Q15).

КАК РАБОТАЕТ

- 01 Подсчёт строк / файлов в diff.
- 02 Если описание PR – много пунктов = много концернов.
- 03 Тесты больше impl – может быть refactor + feature.
- 04 Рекомендация – < 400 LOC, ideally < 200.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR.
- 02 Refactor + новая фича в одном PR.
- 03 Большие миграции.

ПОДВОДНЫЕ КАМНИ

- 01 Reviewer rubber-stamp большой PR.
- 02 Hidden bugs в неосмотренных частях.
- 03 Конфликты при rebase.

МОГУТ ДОСПРОСИТЬ

- 01 Как разбить большой PR?
- 02 Что такое stacked PRs?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/developer/small-cls.html>

Как разбить большой PR?

ОПИСАНИЕ

Стратегии: по слоям (refactor → adapter → feature), stacked PRs (один поверх другого), feature-flag (merge во main, выкатывать постепенно), draft PR с stages. Каждый PR должен быть самодостаточным и пройти CI самостоятельно.

КАК РАБОТАЕТ

- 01 Stacked PRs (graphite, gh tools).
- 02 Feature-flag по умолчанию выключен.
- 03 Refactor сначала, потом feature.
- 04 Каждый PR — green CI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Большие фичи.
- 02 Архитектурные миграции.
- 03 Внедрение новой библиотеки.

ПОДВОДНЫЕ КАМНИ

- 01 Stack ломается при rebase.
- 02 Refactor + feature в одном PR — больно ревьюить.
- 03 Feature-flag забыт = код-долг.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое stacked PRs?
- 02 Как чистить feature-flags?

ПОЛНАЯ СТАТЬЯ

<https://graphite.dev/>

ДЕНЬ 6

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

RUBBER-STAMP**CHECKLIST****КАЧЕСТВО****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Как поднять планку без формализма.

ВОПРОСЫ ДНЯ

Q16 Как бороться с rubber-stamp review?

Q17 Как вводить review checklist?

Q18 Как улучшать code quality в команде?

Как бороться с rubber-stamp review?

ОПИСАНИЕ

Rubber-stamp – формальный approve без реального чтения. Защита: чек-листы (см. Q17), pair-review, минимум 2 approvers на критичные части, ротация ревьюеров, регулярный audit «что попало в main и должно было быть blocker'ом».

КАК РАБОТАЕТ

- 01 Чек-лист на типичные ошибки.
- 02 Pair-review критичных PR.
- 03 Required reviewers по CODEOWNERS.
- 04 Ротация – не один человек ревьюит всё.
- 05 Post-mortem багов: где упустили?

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Команды > 5 разработчиков.
- 02 После production-багов.
- 03 В критичных модулях.

ПОДВОДНЫЕ КАМНИ

- 01 Чек-лист как ритуал, без понимания.
- 02 Слишком жёсткие правила → resentment.
- 03 CODEOWNERS bottleneck.

МОГУТ ДОСПРОСИТЬ

- 01 Что включить в чек-лист?
- 02 Чем pair-review отличается?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/>

ВОПРОС Q17

Как вводить review checklist?

ОПИСАНИЕ

Чек-лист – короткий список (5-10 пунктов) обязательных проверок: тесты, типы, безопасность, ally, перфоманс. Хранится в репозитории (PULL_REQUEST_TEMPLATE.md) и автоматически появляется в каждом PR. Регулярно пересматривать.

КАК РАБОТАЕТ

- 01 .github/PULL_REQUEST_TEMPLATE.md – markdown с пунктами.
- 02 Github автоматически вставляет в новый PR.
- 03 Пересматривать quarterly.
- 04 Чек-лист – для напоминания, не replace мышления.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая команда > 3 разработчиков.
- 02 Junior-heavy команды.
- 03 После регрессий – добавить пункт.

ПОДВОДНЫЕ КАМНИ

- 01 Слишком длинный чек-лист – игнорируется.
- 02 Чек-лист без enforcement – формальность.
- 03 Неактуальные пункты – снижают доверие.

МОГУТ ДОСПРОСИТЬ

- 01 Что включить в base-чек-лист?
- 02 Как обновлять чек-лист?

ПОЛНАЯ СТАТЬЯ

<https://docs.github.com/en/communities/using-templates-to-encourage-useful-issues-and-pull-requests/creating-a-pull-request-template-for-your-repository>

Как улучшать code quality в команде?

ОПИСАНИЕ

Систематически: автоматизация (lint, format, type-check, tests в CI), regular code-review, парные ревью, engineering excellence sessions, обмен знаниями, ADR для решений, post-mortem для инцидентов, dedicated time на refactor.

КАК РАБОТАЕТ

- 01 Pre-commit / CI – lint, type, test.
- 02 Pair-programming для сложных частей.
- 03 Tech-talks – расшаривать находки.
- 04 ADR-журнал решений.
- 05 Refactor-time в каждом sprint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая растущая команда.
- 02 После регрессий.
- 03 Onboarding новых.

ПОДВОДНЫЕ КАМНИ

- 01 «Улучшайте качество» – лозунг без конкретики.
- 02 100% автоматизация без культуры.
- 03 Refactor-tax без бизнес-цели.

МОГУТ ДОСПРОСИТЬ

- 01 Какие метрики качества смотреть?
- 02 Что включить в quality gate?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/articles/agile-architecture.html>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

2 ВОПРОСА

LEGACY**ПЕРФЕКЦИОНИЗМ**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Не блокировать `delivery`, но не пропускать долг.

ВОПРОСЫ ДНЯ

Q19 Как работать с `legacy PR`?Q20 Как не блокировать `delivery` перфекционизмом?

Как работать с legacy PR?

ОПИСАНИЕ

Legacy = старый стиль / нет тестов / устаревшие паттерны. Подход: правильно vs практично – не ломать существующее, boy-scout-rule (оставить чище, чем нашёл), incremental modernization, тесты на новые правки, TODO с tracking.

КАК РАБОТАЕТ

- 01 Boy-scout: чистить рядом с правкой.
- 02 Не переписывать всё в PR-фиксе бага.
- 03 Тесты на новый код.
- 04 TODO с issue-номером для tracking.
- 05 Strangler-pattern для миграций.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Bug-fix в legacy.
- 02 Новая фича в старом модуле.
- 03 Миграции.

ПОДВОДНЫЕ КАМНИ

- 01 Big-bang refactor в bug-fix PR.
- 02 Игнор legacy – никогда не улучшается.
- 03 TODO без tracking – забыто.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое boy-scout rule?
- 02 Когда переписать, а когда оставить?

ПОЛНАЯ СТАТЬЯ

<https://martinfowler.com/bliki/StranglerFigApplication.html>

ВОПРОС Q20

Как не блокировать delivery перфекционизмом?

ОПИСАНИЕ

Принцип: достаточно хорошо > идеально. Чёткие критерии blocker (см. Q2). Suggestions – отдельно. Если код работает, безопасен, протестирован, стилевые улучшения можно отложить. Ревью – не время для архитектурных переписок (это design-doc / ADR).

КАК РАБОТАЕТ

- 01 Blocker / suggestion – явная разница.
- 02 Архитектурные обсуждения – до PR.
- 03 Time-box ревью – не ждать дни.
- 04 Approve с follow-up issue.
- 05 «Не идеально, но merge'абельно».

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой PR.
- 02 Особенно – junior-PR.
- 03 Когда дедлайн.

ПОДВОДНЫЕ КАМНИ

- 01 Каждый nitpick → blocker.
- 02 Силевые войны.
- 03 Дедлайн упущен из-за nitpicks.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое follow-up issue?
- 02 Как разделить вкус от качества?

ПОЛНАЯ СТАТЬЯ

<https://google.github.io/eng-practices/review/reviewer/standard.html>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать иерархию проверок при ревью (Q1)
- ☐ [] Развести blocker и suggestion (Q2, Q3)
- ☐ [] Описать тон фидбека и работу со спорами (Q4, Q5, Q6)
- ☐ [] Расписать ревью архитектуры, hooks, TS (Q7, Q8, Q9)
- ☐ [] Расписать ревью perf, security, ally, tests (Q10-Q13)
- ☐ [] Описать признаки большого PR и стратегии разбивки (Q14, Q15)
- ☐ [] Описать борьбу с rubber-stamp и ввод чек-листа (Q16, Q17)
- ☐ [] Сказать, как улучшать качество и работать с legacy (Q18, Q19)
- ☐ [] Сформулировать баланс между качеством и delivery (Q20)